



現代通信技術導論

统计编码

目录 | CONTENTS

1

什么是数字信息压缩？

2

为什么要进行压缩？

3

信息压缩分类

4

统计编码

什么是数字信息压缩?

三人行必有我师

实事求是

四个意识，四个自信，两个维护

什么是数字信息压缩?

班马

YYDS

ZWR

目录 | CONTENTS

1

什么是数字信息压缩？

2

为什么要进行压缩？

3

信息压缩分类

4

统计编码

为什么要进行数字信息压缩?

从**通信**角度来看:

一路4K电视
数码率
需要



$$I = 4096 \times 2160 \times 60 \times 3 \times 8 \approx 12.7 \text{Gb/s}$$

为什么要进行数字信息压缩?

从**存储**角度来看:

一台5000万像素的
无反相机SONY. α1



α1
50 MP
30 fps
8K 30P/4K 120P

Exmor RS BIONZ XR
EXMOR RS Sensor



为什么要进行数字信息压缩?



$$8688 \times 5792 \times 3 \times 8 / 8 =$$

150MB

SONY. α1 样张

为什么要进行数字信息压缩?

时域压缩

较快地传输各种信源

能量域压缩

降低发射机功率



频域压缩

开通更多的并行业务

空域压缩

减少数据存储容量

目录 | CONTENTS

1

什么是数字信息压缩？

2

为什么要进行压缩？

3

信息压缩分类

4

统计编码

数字信息压缩的分类?

$$\text{数据} = \text{信息} + \text{冗余度}$$

信源熵的基本性质和定理

- 非负性
- 对称性
- 最大离散熵定理
 - 信源 X 中包含 n 个不同离散消息时, 信源熵 $H(X)$ 有 $H(X) \leq \log_2 n$, 当且仅当 X 中各个消息出现的概率全相等时, 上式取等号。
 - 由此可见, 只要信源不是等概率分布, 就存在着数据压缩的可能。

各种熵之间的关系

- 从上表可以看出:
 - 条件熵必定不大于无条件熵,
 - 联合信息熵必不大于各分量信源熵之和。
 - 这两者之差, 就反映了此联合信源所含有的冗余度

$$r = H(X) + H(Y) - H(XY) = I(X; Y)$$

冗余度隐含在信源间的相关性之中, 通常不宜直接对各相关分量进行编码, 尽量先去除各分量间的相关性, 再对各独立分量进行编码。

数字信息压缩的分类?

数据 = 信息 + 冗余度

将茶叶 (**数据**) 倒入一个罐子 (**存储器**)



数字信息压缩的分类?



度

罐子装满后，轻轻晃一晃，肯定可以再多装一些，即在一定程度挤掉空气所占据的存储空间（外在冗余度），使茶叶排列更紧密

数据紧缩

如果原来茶叶受了潮，体积自然膨胀，即“数据”具有（内在冗余度）- 水分。如果烘干去掉水气，那么其体积就会收缩

冗余度压缩

如果将茶叶压碎，显然还可再多装一些，但此时已经“不可逆”

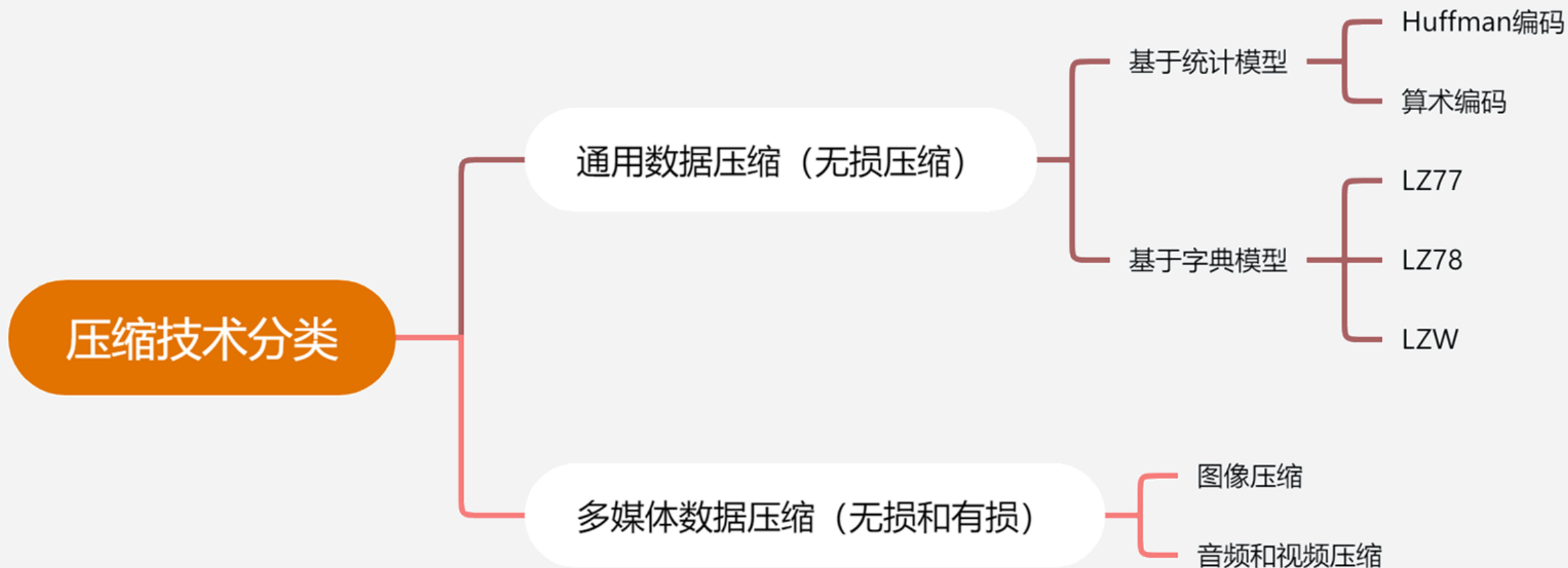
有失真压缩

结论

1. 有冗余度就可以压缩（罐装有空气，茶叶含水分）
2. 压缩只能在一定**限度**内可逆（茶叶倒出来仍然完整）
3. 超过此**限度**，必然带来失真（茶叶会破碎）
4. 允许的失真越大，压缩比例也越高。

这个“度”就是信息的 “**熵**”

数字信息压缩的分类?



目录 | CONTENTS

1

什么是数字信息压缩？

2

为什么要进行压缩？

3

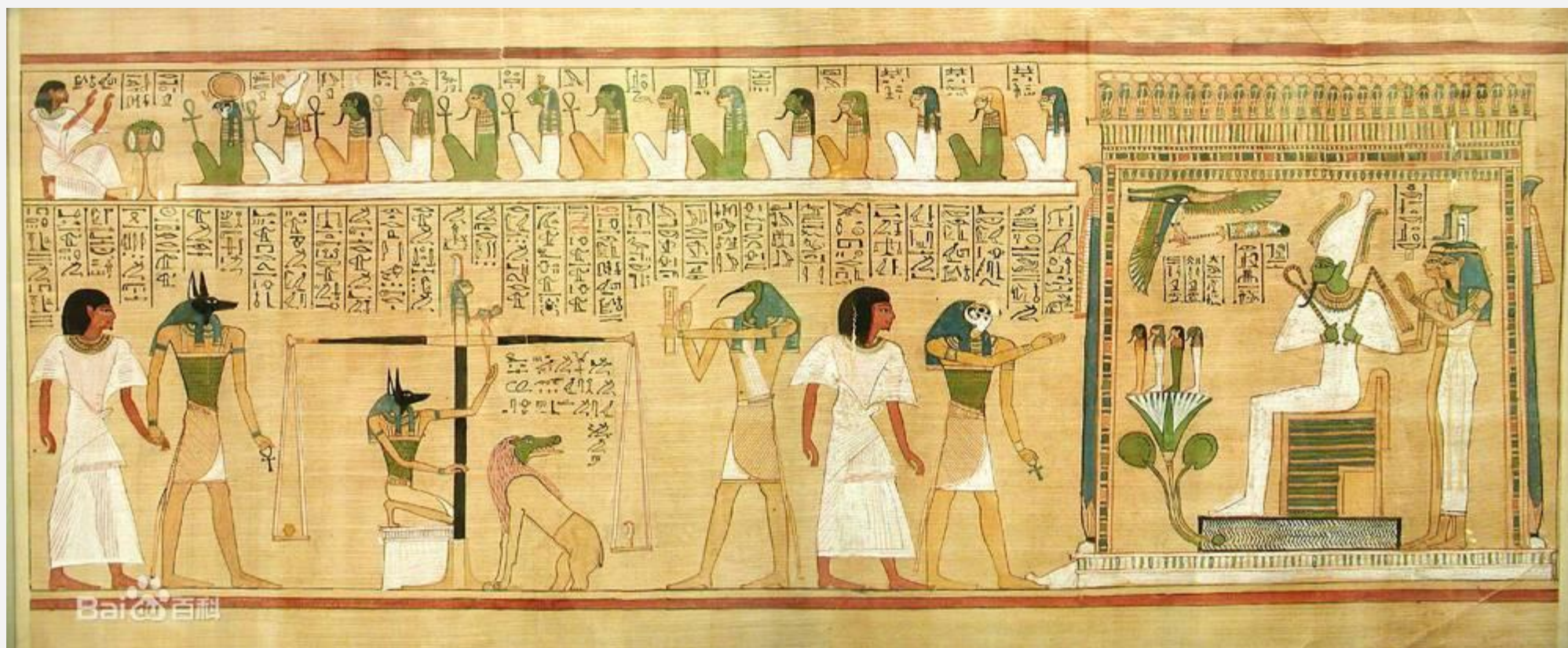
信息压缩分类

4

统计编码

语言和数字是如何产生的







漢代造紙工藝流程圖



.	1
..	2
...	3
....	4
.....	5
.....	6
.....	7
.....	8
.....	9
.....	10

1. 你可以用两只手表示100个数字吗?
2. 那可以表示1000个数字吗?









伸出 伸出 伸出 伸出 伸出 伸出 伸出 伸出 伸出 伸出

收起 收起 收起 收起 收起 收起 收起 收起 收起 收起

统计编码

利用消息或消息序列出现概率的分布特性，
寻找概率与码字长度间的最佳匹配，
叫做**统计编码**或**概率匹配编码**，

统称为**熵编码**。

熵的另一种理解

香农第一定理：无失真信源编码定理

假定有一个信息源，里面有N种信息。

- 需要对这N种信息一一进行编码，如0011表示一种信息，1000111表示第二种，这些编码不能重复，否则就无法根据编码判定是哪种信息。
- 虽然编码可以有好多方法，但是有的方法效率高，有的则效率低，或者说用了很长的编码才能表示一个信息。
- 香农第一定律讲的是，对于信源发出的所有信息，一定存在一种编码方式，使得编码的平均长度无限接近于Ta的信息熵。

它给出了在**无损**情况下，数据压缩的**临界值**，即**信息熵**。

熵的另一种理解

香农第一定理：无失真信源编码定理

比如：用二进制为含有 n 个符号的某条信息编码，
假设符号 F_n 在整条信息中重复出现的概率为 P_n ，

则该符号的 **熵** 也表示编码该符号所需的 **位数长度**

$$E_n = -\log_2(P_n)$$

整条信息的熵也即表示整条信息所需的位数为： $E = \sum E_n$

举个例子

字符串: a a b b a c c b a a

- 字符串长度为 10, 字符 a, b, c 分别出现了 5, 3, 2 次, 则 a b c 在信息中出现的概率分别为 0.5, 0.3, 0.2,

$$E_a = -\log_2(0.5) = 1 \text{ bit}$$

$$E_b = -\log_2(0.3) = 1.737 \text{ bit}$$

$$E_c = -\log_2(0.2) = 2.322 \text{ bit}$$

- 整条信息的熵也即表达整个字符串需要的位数为:

$$E = E_a \times 5 + E_b \times 3 + E_c \times 2 = 14.855 \text{ bit}$$

举个例子

字符串: a a b b a c c b a a

高四位 低四位		ASCII非打印控制字符										ASCII 打印字符												
		0000					0001					0010		0011		0100		0101		0110		0111		
		0					1					2		3		4		5		6		7		
		+进制	字符	ctrl	代码	字符解释	+进制	字符	ctrl	代码	字符解释	+进制	字符	+进制	字符	+进制	字符	+进制	字符	+进制	字符	+进制	字符	ctrl
0000	0	0	BLANK NULL	^@	NUL	空	16	▶	^P	DLE	数据链路转意	32		48	0	64	@	80	P	96	`	112	p	
0001	1	1	☺	^A	SOH	头标开始	17	◀	^Q	DC1	设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q	
0010	2	2	☺	^B	STX	正文开始	18	↕	^R	DC2	设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r	
0011	3	3	♥	^C	ETX	正文结束	19	!!	^S	DC3	设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s	
0100	4	4	◆	^D	EOT	传输结束	20	¶	^T	DC4	设备控制 4	36	\$	52	4	68	D	84	T	100	d	116	t	
0101	5	5	♣	^E	ENQ	查询	21	♫	^U	NAK	反确认	37	%	53	5	69	E	85	U	101	e	117	u	
0110	6	6	♠	^F	ACK	确认	22	■	^V	SYN	同步空闲	38	&	54	6	70	F	86	V	102	f	118	v	
0111	7	7	●	^G	BEL	震铃	23	↕	^W	ETB	传输块结束	39	'	55	7	71	G	87	w	103	g	119	w	
1000	8	8	◼	^H	BS	退格	24	↑	^X	CAN	取消	40	(	56	8	72	H	88	X	104	h	120	x	
1001	9	9	○	^I	TAB	水平制表符	25	↓	^Y	EM	媒体结束	41	)	57	9	73	I	89	Y	105	i	121	y	
1010	A	10	◻	^J	LF	换行/新行	26	→	^Z	SUB	替换	42	*	58	:	74	J	90	Z	106	j	122	z	
1011	B	11	♂	^K	VT	竖直制表符	27	←	^[	ESC	转意	43	+	59	;	75	K	91	[	107	k	123	{	
1100	C	12	♀	^L	FF	换页/新页	28	└	^\	FS	文件分隔符	44	,	60	<	76	L	92	\	108	l	124		
1101	D	13	♪	^M	CR	回车	29	↔	^]	GS	组分分隔符	45	-	61	=	77	M	93	]	109	m	125	}	
1110	E	14	🎵	^N	SO	移出	30	▲	^_	RS	记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~	
1111	F	15	☼	^O	SI	移入	31	▼	^-	US	单元分隔符	47	/	63	?	79	O	95	_	111	o	127	Δ	^Back space

统计模型

- 从上面的描述，要压缩一条信息，首先要分析清楚信息中每个符号出现的概率。
- 不同的压缩程序通过不同的方法确定符号的出现概率，对符号的概率计算得越准确，也就越容易得到好的压缩效果。
- 在压缩程序中，用来处理输入信息，计算符号的概率并决定输出哪个或哪些代码的模块叫做**模型**。

难道信息中字符的出现概率很难估计，
以至于有各种不同的压缩模型吗？

静态统计模型

- 如果预先扫描文件中的所有字符，统计出每个字符出现的概率，这种方法在压缩术语里叫做“**静态统计模型**”。
- 不同的文件中，字符有不同的分布概率，我们要么先花上大量的时间统计要压缩的所有文件中的字符概率，要么为每一个单独的文件保存一份概率表以备解压缩时需要。

静态统计模型 → 自适应模型

自适应模型

- **自适应模型**可以说是一台学习机。
- Ta在信息被输入之前对信息内容一无所知，假定每个字符的出现概率均等，随着字符不断被输入和编码，Ta统计并记录已经出现过的字符的概率并将这些概率应用于对后续字符的编码。
- 自适应模型在压缩开始时压缩效果并不理想，但随着压缩的进行，Ta会越来越接近字符概率的准确值，并达到理想的压缩效果。

统计模型

上面提到的模型可以统称为【统计模型】，
Ta们都是基于对每个字符出现次数的统计得到字符概率的。

你能举出一个统计模型的例子么？



MORSE CODE

A • -

B - • • •

C - • - •

D - • •

E •

F • • - •

G - - •

H • • • •

I • •

J • - - -

K - • -

L • - • •

M - -

N - •

O - - -

P • - - •

Q - - • -

R • - •

S • • •

T -

U • • -

V • • • -

W • - -

X - • • -

Y - • - -

Z - - • •





SOS

国际莫尔斯电码救难信号

并非任何单字的缩写

在野外，你可以用手电筒发出SOS信号吗？

Morse Code – Hello World



编码

通过**模型**，已经确定了对某一个符号该用多少位二进制数进行编码。



如何设计一种**编码**，使其尽量精确地用计算出来的位数表示某个符号？

编码

如果对 a 用 3 个二进制位就表示，
对 b 用 4 个二进制位表示，
在解码时，
面对一连串的二进制流，

100100010011011100111100110101001110000111111111

怎么知道哪3个是 a，哪4个是 b 呢？

编码

前缀编码

任何一个字符的编码，都不是另一个字符编码的前缀。

举个例子

符号	编码
A	0
B	10
C	110
D	1110
E	11110

1110010101110110111100010 是什么信息?

DABBDCEAAB

总结

$$\text{压缩} = \text{模型} + \text{编码}$$

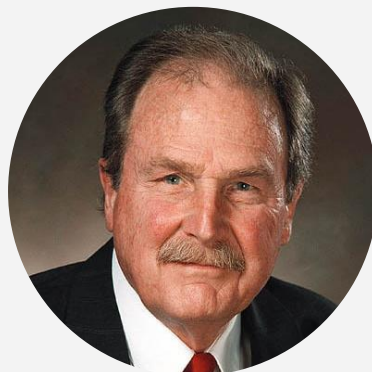
- 不同的**模型**使用不同的方法统计字符的出现概率，由此概率可以得出字符的熵；
- 然后使用不同的**编码**，尽量接近期望熵值。
- 所以，压缩效果的好坏一方面取决于模型能否准确地得到字符概率，另一方面也取决于编码方法能否准确地用期望的位数输出字符代码。

为什么是二叉树?

提到Huffman ,
至少会联想到二叉树和二进制编码,

...

Huffman 编码



David Albert Huffman

为什么是二叉树?

压缩 = 模型 + 编码

信息压缩必须全面考虑模型和编码两个模块的功效;
但同时, 两个模块又相互具有独立性。

A模型

B模型

+

Huffman编码

C模型

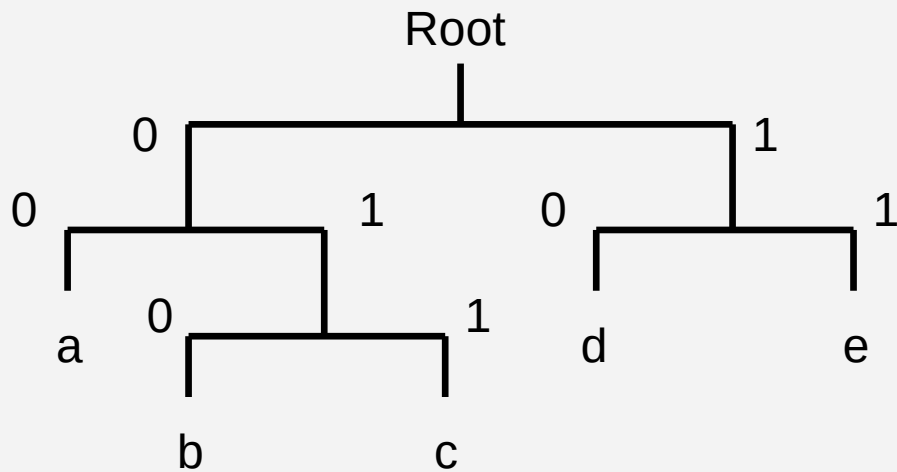
为什么是二叉树?

为什么
信息压缩中的编码方法
总和
二叉树联系在一起呢?

为了使用不固定的码长表示单个字符，编码必须符合“**前缀编码**”的要求，即较短的编码决不能是较长编码的前缀。

为什么是二叉树?

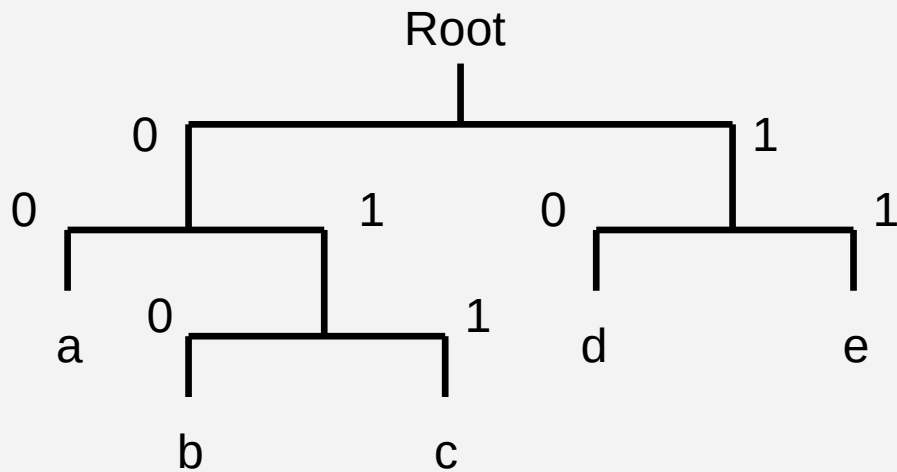
- 符合这一要求的二进制编码体系，二叉树是最理想的选择。



要编码的字符总是出现在树叶上，
假定从根向树叶行走的过程中，
左为0，右为1，
则一个字符的编码就是
从根走到该字符所在树叶的路径。

为什么是二叉树?

- 字符只能出现在树叶上, 任何一个字符的路径都不会是另一字符路径的前缀路径



a - 00

b - 010

c - 011

d - 10

e - 11

Shannon-Fano编码



Claude Shannon



Robert M. Fano

Shannon-Fano 编码

Shannon-Fano编码

例：对下面这串出现了五种字符的信息进行编码(40个字符)

cabcedacacdeddaaabaabababaaabbacdebaceada

答：五种字符的出现次数分别

a - 16, b - 7, c - 6, d - 6, e - 5

Shannon-Fano编码

- S-F编码核心是构造二叉树

1. 将给定符号按照频率从大到小排序
2. 将序列分成上下两部分，使得上部频率总和尽可能接近下部频率总和
3. 把第2步中划分出的上部作为二叉树的左子树，记0，下部作为右子树，记1
4. 分别对左右子树重复2,3 两步，直到所有的符号都成为二叉树的树叶为止

a - 16

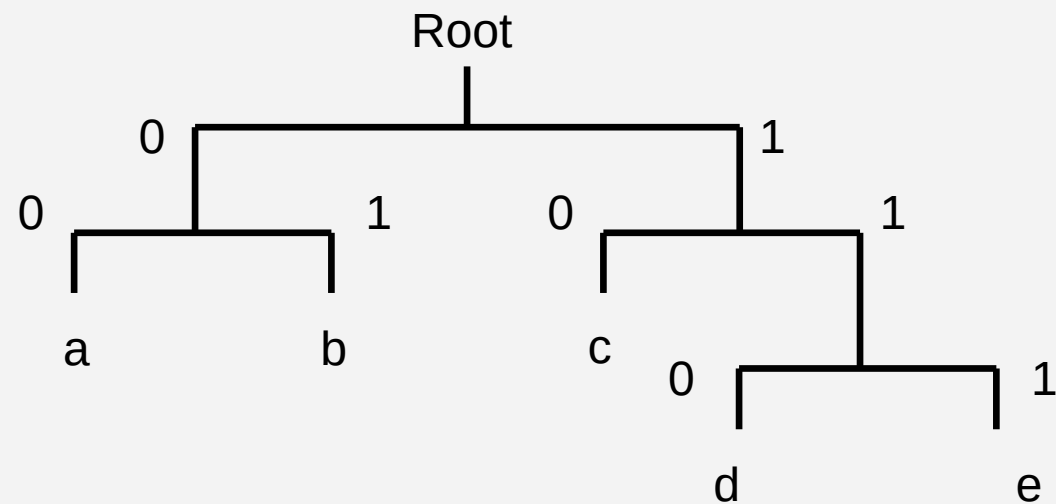
b - 7

c - 6

d - 6

e - 5

Shannon-Fano编码



a - 00

b - 01

c - 10

d - 110

e - 111

Shannon-Fano编码

- 例子中的信息可以编码为：

cabcedeacacdeddaaabaababaaabbacdebaceada

10 00 01 10 111 110 111 00 10 00 10

码长共 91 位

ASCII 码表示上述信息需要 $8 \times 40 = 320$ 位

Huffman 编码

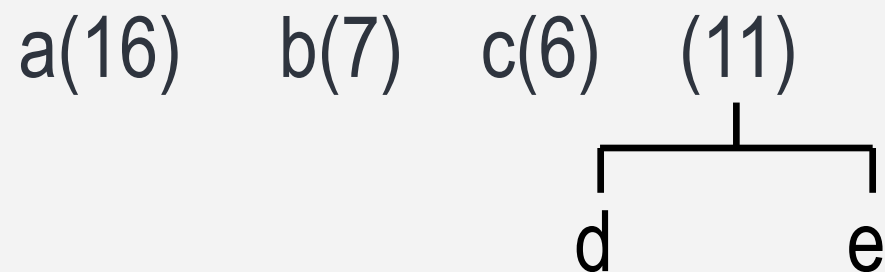
- Huffman 编码构造二叉树的方法和 S-F正好相反，不是自上而下，而是从树叶到树根生成二叉树。

cabcedeacacdeddaaabaababaaabbacdebaceada

Huffman 编码

a(16) b(7) c(6) d(6) e(5)

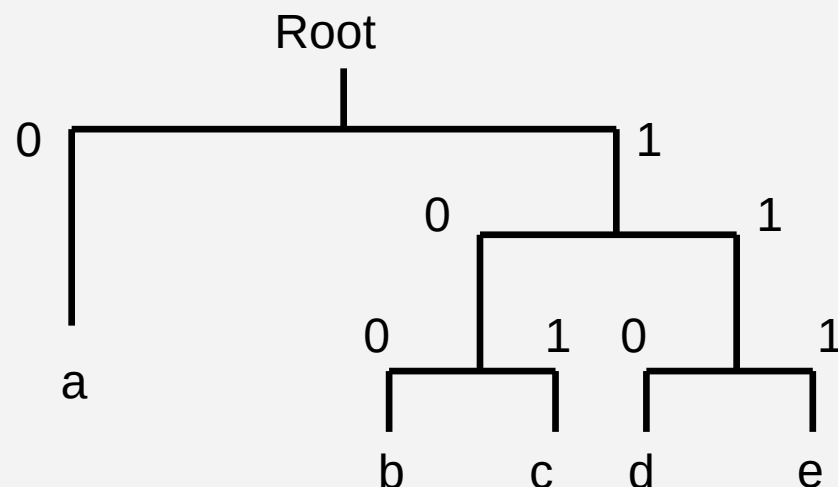
1. 将各个符号及其出现频率分别作为不同的小二叉树（目前每棵树只有根节点）
2. 在 1 中得到的树林里找出频率值最小的两棵树，将Ta们分别作为左、右子树连成一棵大一些的二叉树，该二叉树的频率值为两棵子树频率值之和。



Huffman 编码

a(16) b(7) c(6) d(6) e(5)

3. 对上面得到的树林重复 2 的做法，直到所有符号都连入树中为止。



a - 0 b - 100 c - 101 d - 110 e - 111

Huffman 编码

- 例子中的信息可以编码为：

cabcedeacacdeddaaabaababaaabbacdebaceada

101 0 100 101 111 110 111 0 101 0 101

码长共 88 位

比S-F编码更短

Huffman 编码

- 回顾一下熵的知识，使用已经学到的信源熵计算方法，上面的例子中，每个字符的信息量为：

- $E_a = -\log_2(16 / 40) = 1.322$

- $E_b = -\log_2(7 / 40) = 2.515$

- $E_c = -\log_2(6 / 40) = 2.737$

- $E_d = -\log_2(6 / 40) = 2.737$

- $E_e = -\log_2(5 / 40) = 3.000$

$$E = E_a * 16 + E_b * 7 + E_c * 6 + E_d * 6 + E_e * 5 = 86.601$$

Huffman 编码

符号	理想位数:熵	S-F 编码需要位数	Huffman 编码需要位数
a	1.322	2	1
b	2.515	2	3
c	2.737	2	3
d	2.737	2	3
e	3.000	3	3
总计	86.601	91	88

无论是 S-F还是 Huffman，都只能用近似的整数位来表示单个符号，而不是理想的小数位。

上课练习

- 单符号离散无记忆信源：

$$\left[\frac{X}{P(X)} \right] = \left\{ \begin{matrix} x_1 & x_2 & x_3 & x_4 & x_5 \\ 0.4 & 0.2 & 0.2 & 0.1 & 0.1 \end{matrix} \right\}$$

- 用两种不同的策略对其编二进制Huffman码。
 1. 合并后的新符号排在其他相同概率符号的后面（概率小的位置）
 2.前面（概率大的位置）并求出平均码长。